Campus Resource Hub

A Centralized Web Platform for Academic Resource Sharing with AI-Powered Moderation

A secure and intelligent platform for academic resources and campus collaboration

A project report submitted in partial fulfillment of the Requirements for the Degree of Bachelor of Science in Computer Science & Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NORTHERN UNIVERSITY BANGLADESH

July 2026

APPROVAL

The Project Report "Campus Resource Hub: A Centralized Web Platform for Academic Resource Sharing with AI-Powered Moderation" submitted by Akib Rayhan ID: 42230200858, Rakibul Hasan ID: 42230200859, Mahdi Hasan ID: 42230200717 to the Department of Computer Science and Engineering, Northern University Bangladesh, has been accepted as satisfactory for the partial fulfillment of the requirement for the degree of Bachelor of Science in Computer Science and Engineering and approved as to its style and contents.

Board of Examiners:

1. Tanmoy Mazumder (Supervisor)

2. (Examiner)

3. (Examiner)

Md. Raihan-Ul-Masood
Associate Professor & Head
Department of Computer Science & Engineering
Northern University Bangladesh

DECLARATION

We hereby declare that the work presented in this project report is the outcome of the investigation performed by us under the supervision of Tanmoy Mazumder, Lecturer, Department of Computer Science & Engineering, Northern University Bangladesh. We also declare that no part of this project has been or is being submitted elsewhere for the award of any degree or diploma.

Signature

. . . . . .

Akib Rayhan
ID: 42230200858

. . . . . .

Rakibul Hasan
ID: 42230200859

. . . . . .

Mahdi Hasan
ID: 42230200717

ABSTRACT

This project presents the design and implementation of *Campus Resource Hub*, a centralized full-stack web platform that brings academic resource sharing, campus events, clubs, lost-and-found services, announcements, and real-time communication into a single system for a university community. The platform is built on the MERN-style stack: a React single-page application on the frontend and a Node.js/Express REST API with MongoDB on the back-end. Students can upload and download categorized study materials (PDF, DOCX, PPTX, XLSX, and images), which pass through an automated AI content-safety scan; unsafe, partially inspected, or provider-unavailable uploads enter a human moderation workflow before publication. The system implements secure email-OTP–verified registration, JWT-based session management, and three-tier role-based access control (student, moderator, administrator). Real-time private and group messaging is provided through Socket.io with delivery and seen receipts, and an event scheduler supports capacity-aware RSVP management. A distinctive feature of the platform is its semantic search subsystem: every published item is converted into a 384-dimensional multilingual sentence embedding and indexed in MongoDB Atlas Vector Search, enabling an AI assistant to answer natural-language questions about campus content with streamed responses. Files are stored in Cloudinary, and the system is deployed on cloud infrastructure at zero recurring cost. Testing shows the current free-tier deployment comfortably serves a department-scale user base of 50–100 concurrent active users, with a clear and inexpensive upgrade path to campus scale. The result is a practical, secure, and extensible digital campus platform that reduces information fragmentation and modernizes day-to-day academic collaboration.

ACKNOWLEDGEMENT

First and foremost, we express our sincere gratitude to Almighty God, the Most Merciful, for granting us the strength, patience, and opportunity to complete this project successfully. The development of *Campus Resource Hub* was a demanding but rewarding journey, and its completion would not have been possible without the guidance, encouragement, and support of many people.

We would like to convey our deepest appreciation to our project supervisor, Tanmoy Mazumder, Lecturer, Department of Computer Science and Engineering, Northern University Bangladesh. His thoughtful guidance, constructive criticism, and continuous encouragement helped us refine both the technical implementation and the presentation of this work. His willingness to review our ideas, identify weaknesses, and suggest practical improvements was invaluable throughout the planning, development, testing, and report preparation stages.

We are also sincerely grateful to Md. Raihan-Ul-Masood, Associate Professor & Head of the Department of Computer Science & Engineering, Northern University Bangladesh, for providing the academic direction and supportive environment necessary to carry out this project. We extend our thanks to all respected faculty members of the department whose courses, advice, and classroom instruction gave us the knowledge required to work with software engineering, database systems, web technologies, networking, information security, and applied artificial intelligence. We equally appreciate the administrative and laboratory staff for their cooperation and assistance during our undergraduate studies.

We gratefully acknowledge our classmates, friends, and the students who shared practical observations about resource exchange, campus communication, and usability. Their feedback during informal testing helped us recognize real user needs and improve several workflows of the system. We also thank the developers and maintainers of the open-source technologies and documentation used in this project, which made it possible for us to design and implement a full-stack platform within a student budget.

Finally, we offer our heartfelt gratitude to our parents and family members for their unconditional support, patience, prayers, and motivation throughout our education. We also appreciate one another as team members for sharing responsibilities, resolving challenges, and remaining committed through every stage of the project. We dedicate this work to everyone who supported our academic journey and inspired us to create a system that may contribute meaningfully to the university community.

Contents

List of Figures

List of Tables

Chapter 1: Introduction

University students constantly exchange study materials, event information, club activities, and everyday campus updates. In most institutions this exchange is scattered across social media groups, messaging apps, email threads, and physical notice boards. Important files get buried in chat histories, event announcements never reach everyone, found items rarely return to their owners, and there is no quality control over what is shared. *Campus Resource Hub* (CRH) addresses this fragmentation by providing one moderated, searchable, real-time platform for the entire campus community.

1.1 Background and Motivation

The motivation for this project came from observing how students in our own department share resources today: class notes travel through Facebook Messenger groups, seminar notices are photocopied onto notice boards, and lost ID cards are announced verbally. Each channel reaches only a fraction of students, content disappears quickly, and nothing is verified. A purpose-built platform can solve all of these problems at once, while also applying modern techniques — such as AI content moderation and semantic search — that general-purpose tools do not offer for campus use.

1.2 Problem Statement

The specific problems this project solves are:

1. Study materials have no central, categorized, searchable repository; files are duplicated and lost across chat groups.
2. Campus events lack a unified registration system with capacity control and attendance tracking.
3. Club membership management (joining, approval, member roles) is handled manually.
4. Lost-and-found has no structured claim-and-verify process, and posting personal contact information publicly is unsafe.
5. Announcements cannot be scheduled, prioritized, or tracked for readership.
6. Shared content is completely unmoderated; harmful or inappropriate uploads spread unchecked.

7. Keyword search cannot find content expressed in different words, and there is no way to *ask questions* about campus information.

1.3 Objectives

The objectives of the project are to:

- Build a centralized web platform where verified students can upload, browse, preview, download, and rate categorized study resources.
- Implement secure authentication with email OTP verification, JWT sessions, and role-based access control (student, moderator, admin).
- Provide an automated AI content-safety scan combined with a human approval workflow for all user-generated content.
- Support campus events with capacity-aware RSVP, club management with join policies and member roles, and a privacy-preserving lost-and-found claim system.
- Deliver real-time direct and group messaging with delivery and seen receipts using WebSocket technology.
- Implement semantic (meaning-based) search over all published content using sentence embeddings and vector search, exposed through a streaming AI assistant.
- Deploy the complete system on cloud infrastructure at minimal cost.

1.4 Key Concepts

1.4.1 MERN-Style Web Architecture

The system follows the popular JavaScript full-stack pattern: **M**ongoDB as the document database, **E**xpress as the HTTP framework, **R**eact as the frontend library, and **N**ode.js as the server runtime. The frontend is a single-page application (SPA) that communicates with the backend exclusively through a REST API, keeping the two layers independently deployable.

1.4.2 REST API and JWT Authentication

The backend exposes resources (users, files, events, clubs, etc.) through RESTful endpoints. After login, the server issues a *JSON Web Token* (JWT) — a signed token the client attaches to every request. Middleware verifies the signature and the user's role before allowing access, so protected operations never depend on server-side session storage.

1.4.3 WebSocket and Real-Time Communication

HTTP is request–response; the server cannot push data to a client. *WebSocket* keeps a persistent two-way connection open, and the Socket.io library adds rooms, acknowledgements, and automatic fallback. CRH uses it for instant message delivery, live seen/typing indicators, and real-time notifications.

1.4.4 AI Content Moderation

Every uploaded file's text and images are extracted and sent to a large-language-model moderation service, which classifies content against safety categories (sexual content, violence, harassment, etc.). Flagged uploads are quarantined for human review instead of being published — a two-layer (AI + human) moderation pipeline.

1.4.5 Sentence Embeddings and Semantic Search

An *embedding* maps a piece of text to a numeric vector such that semantically similar texts land near each other in vector space. CRH uses a multilingual MiniLM sentence-transformer model to produce 384-dimensional vectors for every published item, stores them in a dedicated collection, and queries them with MongoDB Atlas Vector Search (k-nearest-neighbour cosine similarity). This lets a search for "exam preparation notes" find a document titled "final term study guide" — something keyword search cannot do.

1.4.6 Cloud Object Storage

User files are not stored on the application server; they are uploaded to Cloudinary, a cloud media service, and the database only keeps secure URLs and public IDs. This keeps the server stateless and lets the platform scale storage independently.

1.5 Report Organization

Chapter 2 reviews existing solutions and positions our proposal. Chapter 3 describes the methodology: architecture, database design, and module design. Chapter 4 gives an implementation overview of every module. Chapter 5 presents results and analysis, including a capacity study. Chapter 6 lists future enhancements, and Chapter 7 concludes the report.

Chapter 2: Related Previous Work

2.1 Discussion on Existing Solutions

Several categories of existing tools partially address campus information sharing. Each category solves a useful part of the problem, but none provides a complete student-centered campus hub.

Social media groups (Facebook Groups, Messenger, WhatsApp). These are the de-facto standard in most universities because they are free and familiar. However, they mix academic content with unrelated posts, provide no file categorization (a lecture note from last semester is effectively unfindable), have no moderation tailored to academic norms, and expose members' personal profiles. Search is normally limited to message text, so students must remember who posted a file or approximately when it appeared. Ownership and moderation also depend on informal group administrators rather than institutional roles.

Learning Management Systems (Moodle, Google Classroom). LMS platforms organize course content well, but they are teacher-centric: students cannot freely share peer resources, and campus-life features — clubs, events with RSVP, lost-and-found, peer chat — are out of scope. Institutional LMS deployments also carry licensing and administration overhead. Their course-by-course structure is effective for formal teaching, yet it fragments cross-department resources and does not provide a unified discovery experience for the wider campus community.

Generic cloud drives (Google Drive, OneDrive). Shared folders solve storage but not discovery: there is no rating, no moderation, no metadata like course/semester, and links rot as owners graduate. Access is often controlled through manually shared links, which makes permission management inconsistent and gives students little confidence that a document is current, safe, or academically relevant.

University notice boards and websites. Official channels are one-directional, updated slowly, and offer no interactivity or personalization. They remain valuable for authoritative notices, but usually do not support student contributions, targeted department feeds, event registration, real-time updates, or searchable archives.

Specialized campus applications. Some institutions deploy separate systems for events, library services, student clubs, or lost-and-found reporting. Such applications provide deeper support for one workflow, but students must maintain several accounts and move between disconnected interfaces. Data produced in one service cannot normally improve search or recommendations in another.

2.2 Limitations Identified in Previous Work

The review reveals five recurring gaps. First, information is **fragmented**: academic files, events, notices, and community activities live in separate channels. Second, discovery is mostly **keyword- or time-dependent**; users cannot ask a natural language question and retrieve relevant passages from inside files. Third, peer-contributed content often lacks a reliable **safety and approval process**. Fourth, the systems provide inconsistent **privacy and role controls**, especially for personal contact information and student-generated posts. Finally, many products omit the **interactive campus workflows** that connect information with action, such as registering for an event, joining a club, claiming a found item, or messaging another user.

These gaps are not independent. For example, adding cloud storage without metadata improves availability but not discovery; adding a search box without moderation may make unsafe material easier to find; and publishing notices without notifications does not ensure that the intended students see them. A useful campus platform must therefore treat storage, retrieval, safety, authorization, and interaction as parts of one architecture.

2.3 Comparison with the Proposed System

Campus Resource Hub combines the useful parts of each category into one student-centric platform and adds capabilities none of them provide in this context:

- **Structured resource repository** with course, department, and semester metadata, ratings, view/download analytics, and in-browser preview.
- **Fail-safe two-layer moderation**: an automated AI safety scan at upload time, with unsafe, partial, or provider-unavailable checks held for moderator review.
- **Campus-life modules** (events with capacity-aware RSVP, clubs with join policies and officer roles, claim-based lost-and-found with private contact reveal) that LMS platforms do not cover.
- **Semantic search and an AI assistant** over all campus content, using sentence embeddings and vector search — beyond the keyword search of every surveyed alternative.
- **Zero-cost deployment** on free cloud tiers, making the system realistic for a public university budget.

Table 2.1 summarizes the functional positioning. "Yes" means that the capability is a normal part of the category; "Limited" means that it depends on manual setup, separate tools, or basic keyword matching.

Table 2.1: Comparison of related solutions with Campus Resource Hub

Capability	**Social media**	**LMS**	**Cloud drive**	**Notice site**	**CRH**
Structured academic metadata	No	Yes	Limited	No	**Yes**
Student resource contribution	Yes	Limited	Yes	No	**Yes**
Automated safety moderation	No	No	No	No	**Yes**
Semantic file-content search	No	Limited	Limited	No	**Yes**
Events, clubs, and lost-found	Limited	No	No	Limited	**Yes**
Role-aware privacy controls	Limited	Yes	Limited	No	**Yes**
Real-time chat and alerts	Yes	Limited	Limited	No	**Yes**
Numbered AI answer sources	No	No	No	No	**Yes**

2.4 Research Positioning and Contribution

The proposed work is not intended to replace an LMS or an official university website. Instead, it occupies the space between formal teaching systems and informal student communication. It preserves the low-friction contribution model of social platforms, the organization of an LMS, the storage flexibility of cloud drives, and the authority controls of institutional systems within one role-aware application.

Its principal contribution is the integration of these established ideas with two applied-AI workflows. The first inspects uploaded text and images before publication and fails safely to human review when automatic inspection is incomplete. The second indexes metadata and ordered passages from PDF, Office, and safe image resources, enabling grounded questions and summaries with visible sources. Combining these workflows with events, clubs, lost-and-found, notifications, and real-time messaging makes the system broader than a document repository while remaining focused on everyday university use.

This comparison also defines the evaluation criteria used later in the report: discoverability, publication safety, authorization, workflow completeness, responsiveness, and deployment cost. The Result and Analysis chapter evaluates the implemented platform against these criteria rather than claiming that any single existing product is universally inadequate.

Chapter 3: Methodology

3.1 System Design

3.1.1 System Architecture

The platform uses a three-tier architecture (Figure 3.1). The presentation tier is a React SPA served from Vercel's CDN. The application tier is a Node.js/Express server on Render that exposes the REST API and the Socket.io gateway. The data tier consists of MongoDB Atlas (documents and vector index), Cloudinary (file objects), and external service providers for email (Brevo SMTP) and AI moderation (Groq).

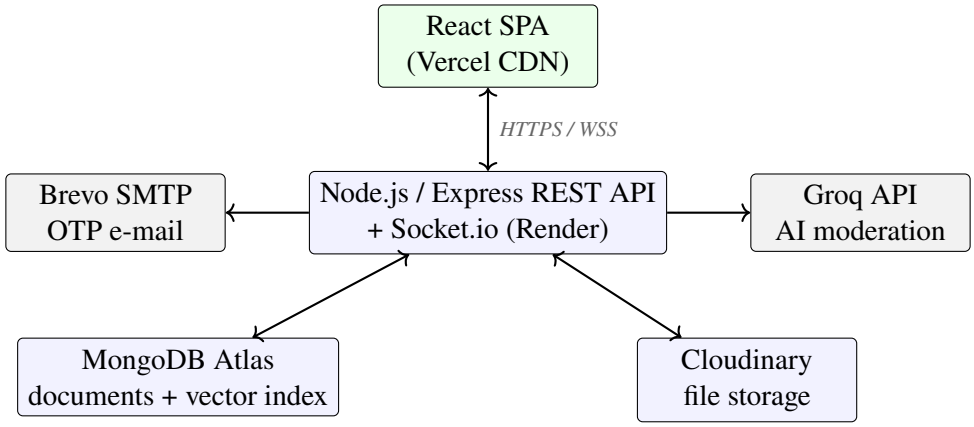

Figure 3.1: Three-tier system architecture of Campus Resource Hub

3.1.2 System Workflow

Figure 3.2 shows the complete system workflow. After OTP-verified signup and JWT login, users reach a role-aware dashboard from which all eleven functional flows branch out: resources, events, clubs, lost-and-found, announcements, chat, notifications, AI search, admin panel, and profile management. The central design pattern shared by all content flows is *create → AI scan → moderator approval → publish → notify*.

3.1.3 User Roles

Access control follows a three-tier role model. Table 3.1 summarizes what each role is allowed to do; every privileged API route enforces these rules through the `authorize` middleware.

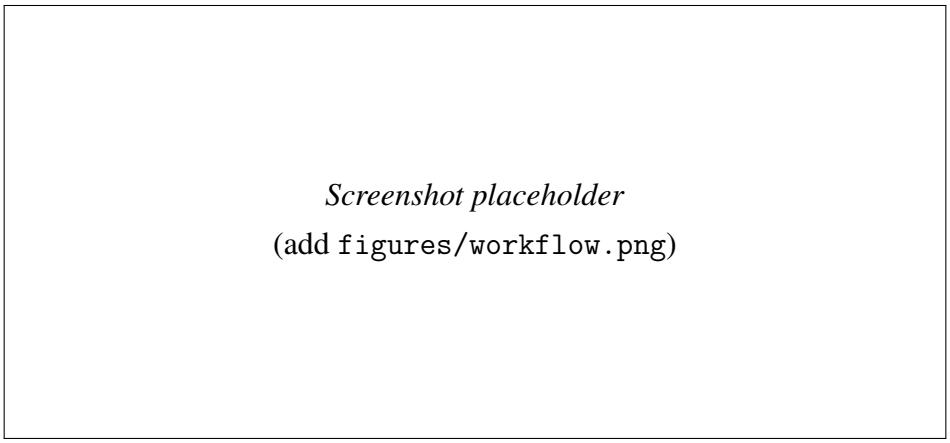

Figure 3.2: Overall system workflow (exported from the project's Lucidchart/draw.io diagram)

Table 3.1: User roles and their permissions

Role	**Permissions**
Student	Browse/search all published content; upload resources; post lost-and-found items; RSVP to events; join clubs; chat; rate and download resources; manage own profile.
Moderator	All student permissions, plus: approve/reject resources, events, announcements, and lost-and-found posts; review AI-flagged items; create events, clubs, and announcements; block/unblock users.
Admin	All moderator permissions, plus: change user roles; delete users; full platform statistics.

3.2 Technology Stack

The complete technology stack is listed in Table 3.2. All layers use JavaScript, which keeps the codebase uniform and allows shared conventions between frontend and backend.

Table 3.2: Technology stack of Campus Resource Hub

Layer	**Technology**	**Purpose**
Frontend	React 18 + Vite	Single-page application UI
	React Router	Client-side routing, protected routes
Backend	Node.js + Express 5	REST API server
	Socket.io	Real-time messaging and notifications
Database	MongoDB Atlas + Mongoose	Document storage, schemas, indexes
	Atlas Vector Search	384-dim semantic search index
AI	Groq LLM API	Content-safety moderation
	Transformers.js (MiniLM)	Local sentence-embedding generation
Storage	Cloudinary	File and image hosting
Auth	JWT + bcrypt	Stateless sessions, password hashing
E-mail	Brevo SMTP + Nodemailer	OTP delivery
Hosting	Vercel / Render	Frontend CDN / backend server

3.3 Database Design

The database consists of eleven MongoDB collections. Their fields and relationships are shown in the entity–relationship diagram (Figure 3.3) and summarized in Table 3.3. The `User` collection is the hub: nearly every other collection references it. All content collections share a common moderation sub-document (`approved`, `approvedBy`, `rejectionReason`, AI verdict), which keeps the approval workflow uniform across modules.

Screenshot placeholder
(add `figures/erd.png`)

Figure 3.3: Entity–relationship diagram of the eleven MongoDB collections (exported from draw.io)

Table 3.3: Summary of database collections

Collection	**Purpose and key fields**
User	Account data: name, unique e-mail and student ID, department, role, bcrypt-hashed password, block status.
AuthOtp	Hashed signup / password-reset OTPs with TTL index for automatic expiry.
Resource	Study material metadata: course, department, semester, file info, Cloudinary IDs, moderation verdict, downloads, rating.
Announcement	Notice content, priority, scheduled `publishAt`, expiry, read receipts, attachments.
Event	Date/time/location, capacity (0 = unlimited), RSVP registrations (going / maybe / declined), club reference.
Club	Name, category, join policy (open / request), members with roles (member / officer), pending join requests.
LostFoundItem	Item type (lost / found), private contact info, claim list with statuses, resolution timestamps.
Conversation	Direct or group chat thread: members, admins, last message reference.
Message	Sender, text, attachments, reply reference, delivered-to and seen-by lists.
Notification	Per-user feed entries with type, link, and read flag.
Embedding	384-dimensional vector, source type + ID (polymorphic), content hash to skip unnecessary re-embedding.
ResourceChunk	Ordered text or safe-image-description passage, parent resource, passage type, and 384-dimensional embedding.

3.4 API Design

The REST API is organized into nine route groups, protected by two middleware layers: `protect` (JWT verification) and `authorize` (role check).

Table 3.4: REST API route groups (representative endpoints)

Group	Base path	Representative operations
Auth	`/api/auth`	signup, verify-signup-otp, login, forgot/reset password, profile update
Resources	`/api/resources`	upload, list/filter, preview, file stream, download count, rate
Announcements	`/api/announcements`	create, approve, pin, mark read, attachment stream
Events	`/api/events`	create, approve, register, RSVP update, cancel
Clubs	`/api/clubs`	create, join/leave, decide request, set member role
Lost-Found	`/api/lost-found`	post, claim, decide claim, resolve, reopen
Chat	`/api/chat`	conversations, messages, group management, AI assistant (stream)
Notifications	`/api/notifications`	list, mark read, clear
Admin	`/api/admin`	statistics, user list, role change, block, delete

3.5 Security Design

Security measures are applied at every layer:

- **Account security**: e-mail ownership proven by OTP before account creation; OTPs stored only as hashes with automatic TTL expiry and attempt counting; passwords hashed with bcrypt (salt rounds = 10).
- **Transport / headers**: HTTPS everywhere, Helmet security headers, strict CORS allow-list.
- **Abuse prevention**: express-rate-limit (1,500 requests per 15 minutes per IP); upload size and type validation with Multer.
- **Authorization**: JWT verification middleware plus role checks on every privileged route; chat access verified per conversation membership.
- **Content safety**: AI moderation scan on every upload; flagged items quarantined; human approval required before anything is published.

- **Privacy**: lost-and-found contact details hidden by default and revealed only to approved claimants; passwords excluded from all API responses.

Chapter 4: Implementation Overview

After a successful login, every user lands on a role-aware dashboard (Figure 4.1) that summarizes platform activity — total resources, announcements, upcoming events, and open lost-and-found items — and provides quick access to recent resources and the latest announcements. The persistent left navigation and the top "Ask AI" search bar are available from every page.

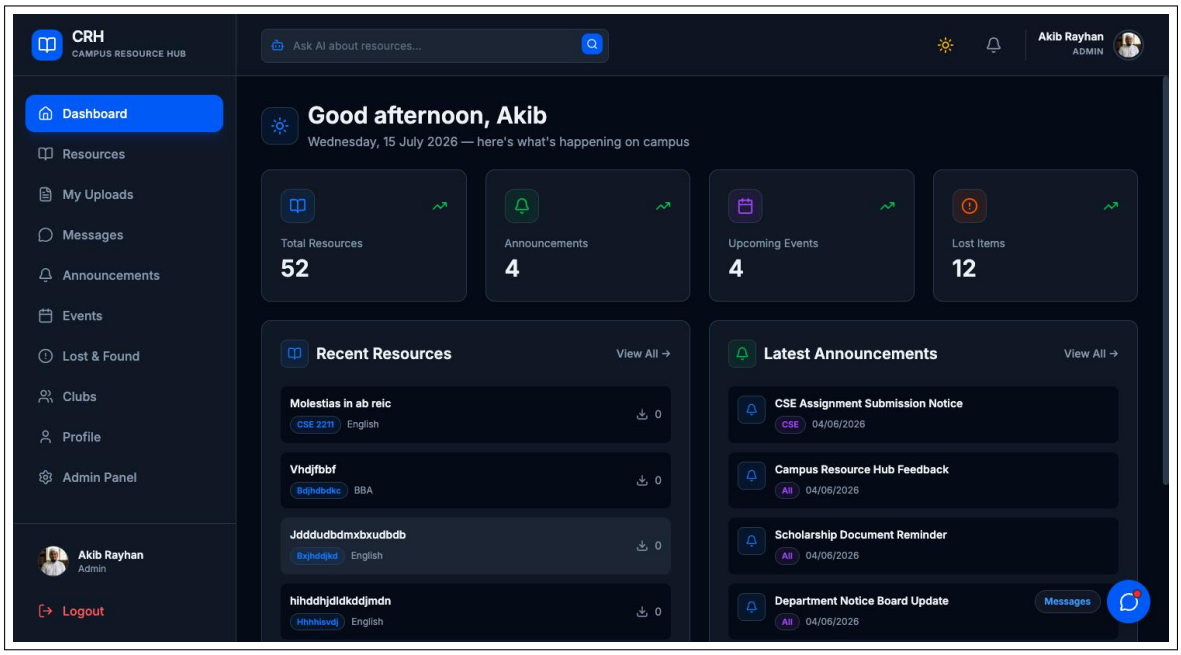

Figure 4.1: Role-aware dashboard with activity summary cards, recent resources, and latest announcements

4.1 Authentication and OTP Verification

Signup collects name, e-mail, student ID, department, and password. The server generates a six-digit OTP, stores its hash in the `AuthOtp` collection with a TTL expiry, and e-mails it via Brevo SMTP. Only after correct OTP entry is the account created with the default `student` role. Login returns a signed JWT; a parallel OTP flow handles forgotten passwords.

4.2 Resource Upload and AI Moderation Pipeline

Upload accepts PDF, DOCX, PPTX, XLSX, and images with course, department, and semester metadata. The file goes to Cloudinary; text and images are then extracted server-side (pdf-parse, Mammoth, canvas) and scanned through a provider chain: Groq first, Hug-

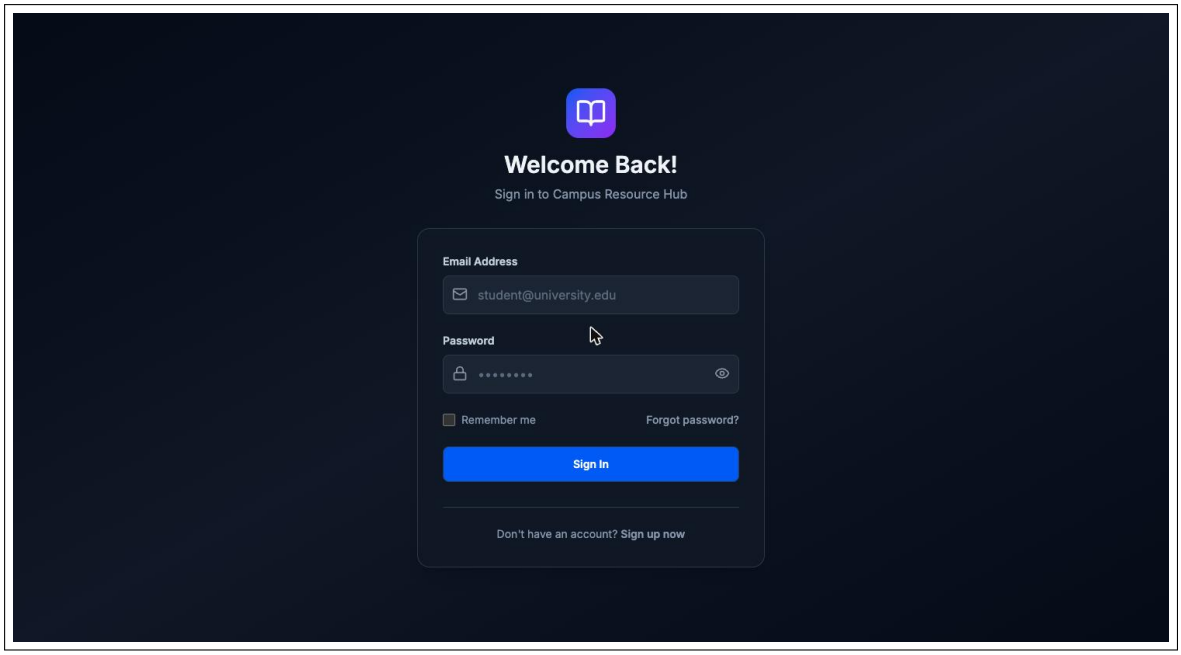

Figure 4.2: Login and signup with e-mail OTP verification

ging Face second, and OpenAI last. Clean, fully inspected uploads are published automatically. Flagged files, partial extractions, and provider-unavailable checks fail safely into the moderator review queue; they are never auto-published. Rejection notifies the uploader with a reason. Searchable passages are generated asynchronously for approved text and safe image descriptions.

4.3 Events with Capacity-Aware RSVP

Moderators create events with date, time, location, and optional capacity. Students RSVP as *going*, *maybe*, or *declined*; registration is blocked once capacity is reached. Users see their events in a calendar view, and posters or admins can cancel events, which notifies registrants.

4.4 Clubs with Join Policies and Member Roles

Each club declares a join policy: *open* (instant membership) or *request* (application with an optional note, decided by an officer or the creator). Members hold *member* or *officer* roles; officers manage requests and membership.

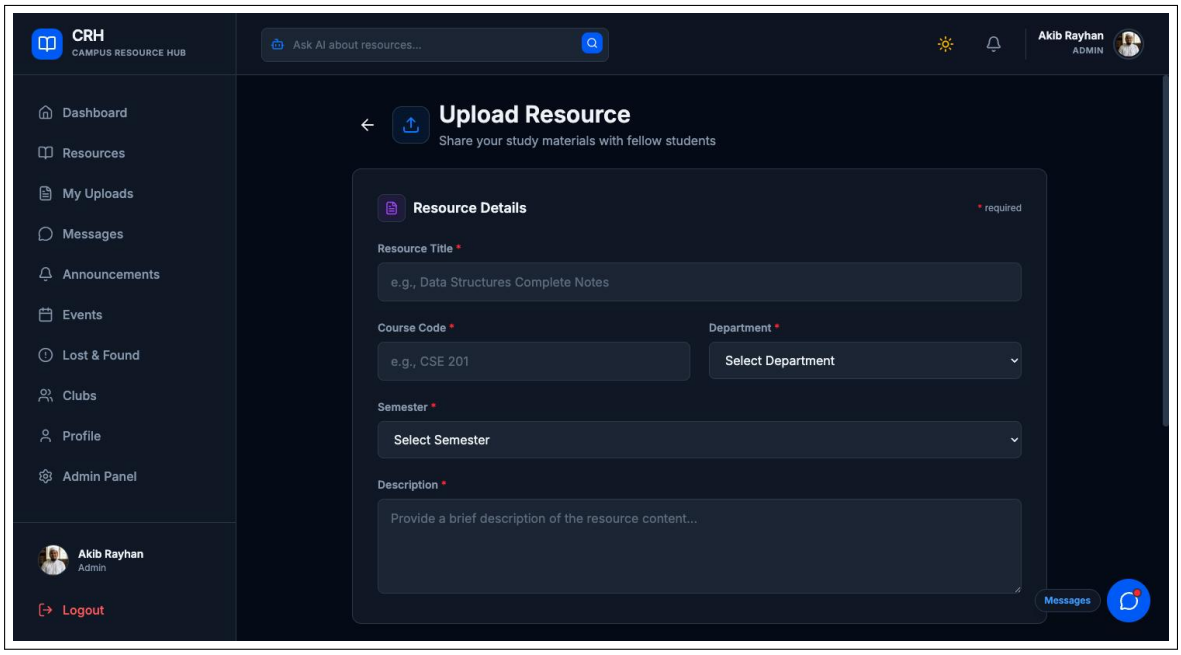

Figure 4.3: Resource upload with metadata and moderation status

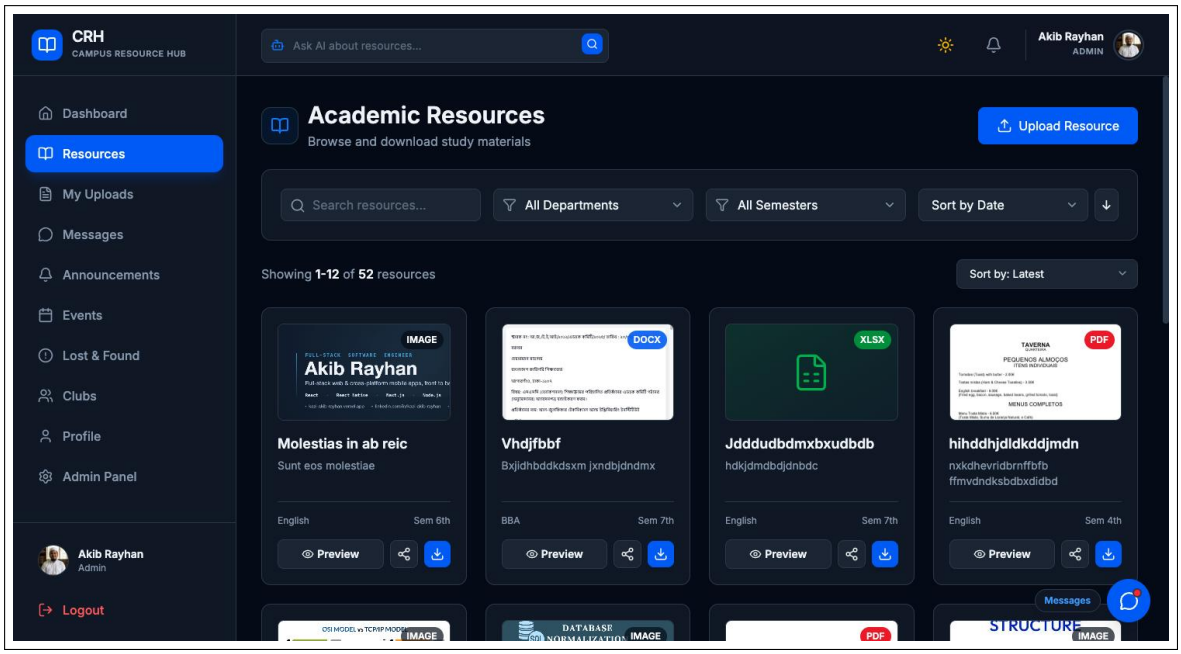

Figure 4.4: Browsing, filtering, previewing and rating published resources

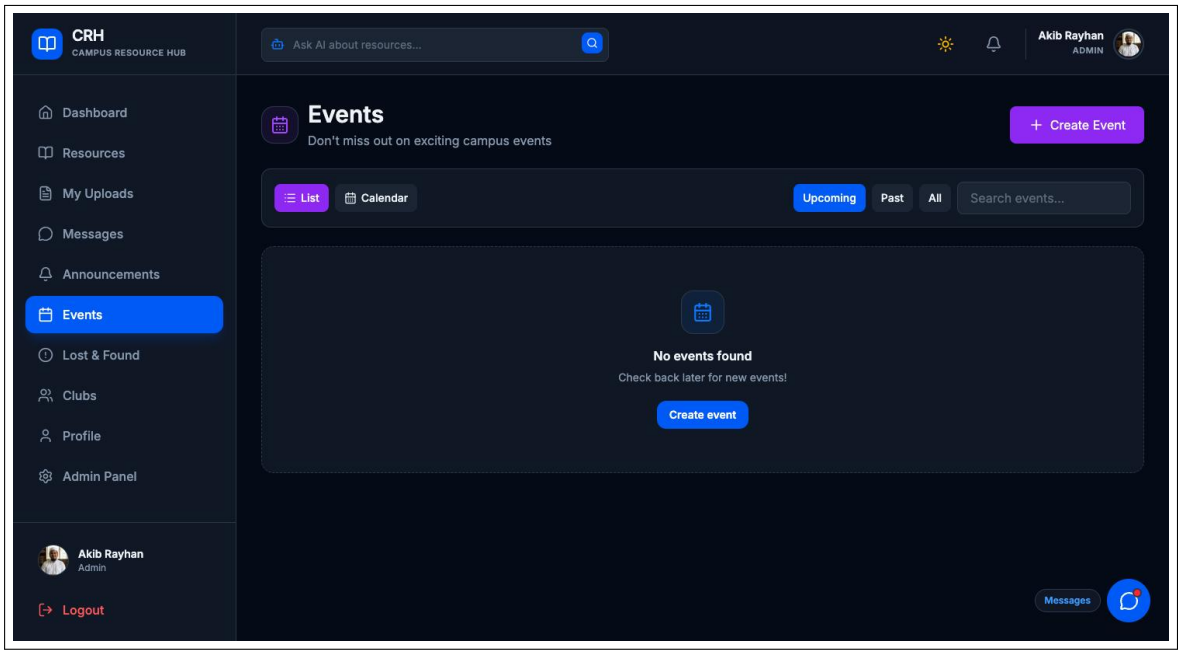

Figure 4.5: Event listing and RSVP flow

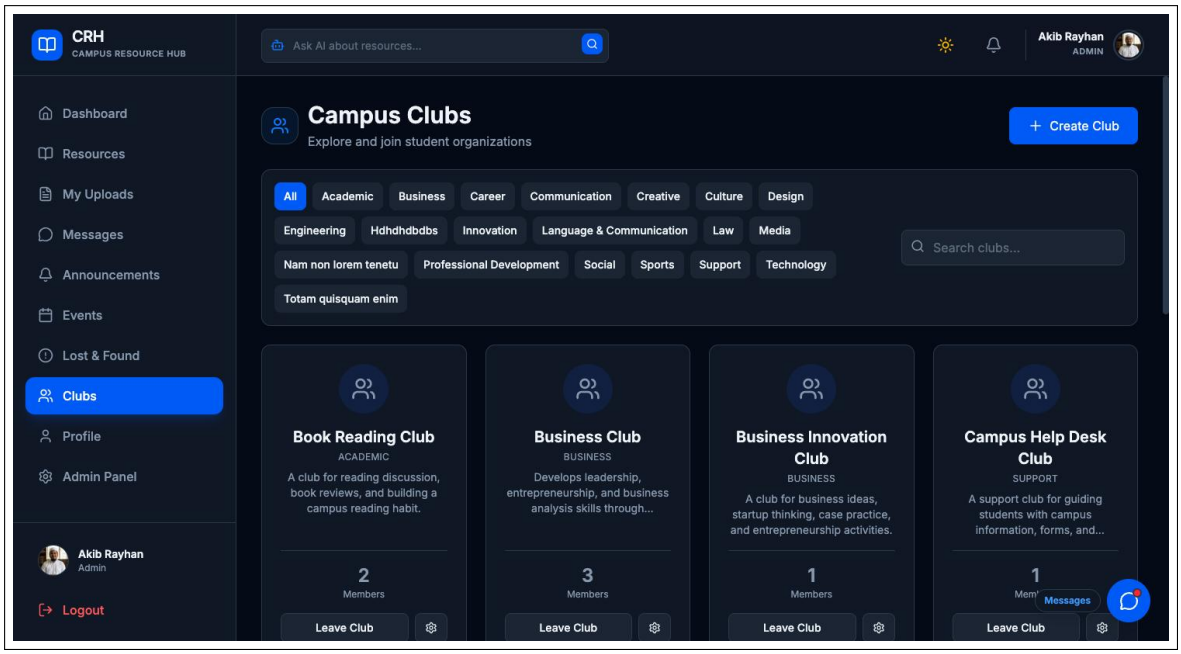

Figure 4.6: Club directory and membership management

4.5 Lost and Found with Claim Workflow

Users post lost or found items with photos. Contact information is hidden by default. Another user files a claim with a note; the poster (or a moderator) approves or rejects it. Approval reveals contact details to the claimant only and moves the item to *claimed*, then *resolved*. This protects personal data while still reuniting items with owners.

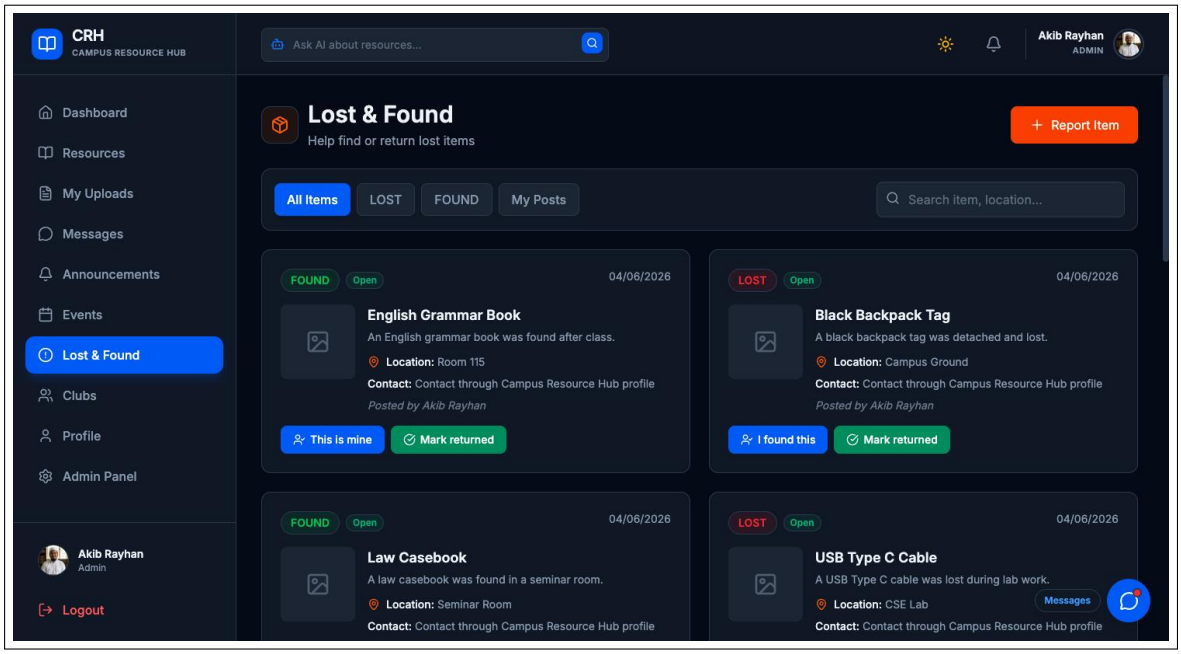

Figure 4.7: Lost-and-found listing and claim dialog

4.6 Announcements with Scheduled Publishing

Announcements carry a priority (normal / important / urgent), optional attachments, a scheduled `publishAt` time, and an optional expiry. A server-side scheduler publishes each announcement exactly once at its scheduled time and broadcasts a notification. Read receipts record which students have seen each notice.

4.7 Real-Time Chat

Chat supports direct and group conversations over Socket.io. Messages support attachments, replies, and editing; the system tracks per-user delivery and seen status. Group admins can rename groups, add or remove members, and promote other admins. All chat REST endpoints re-verify conversation membership server-side.

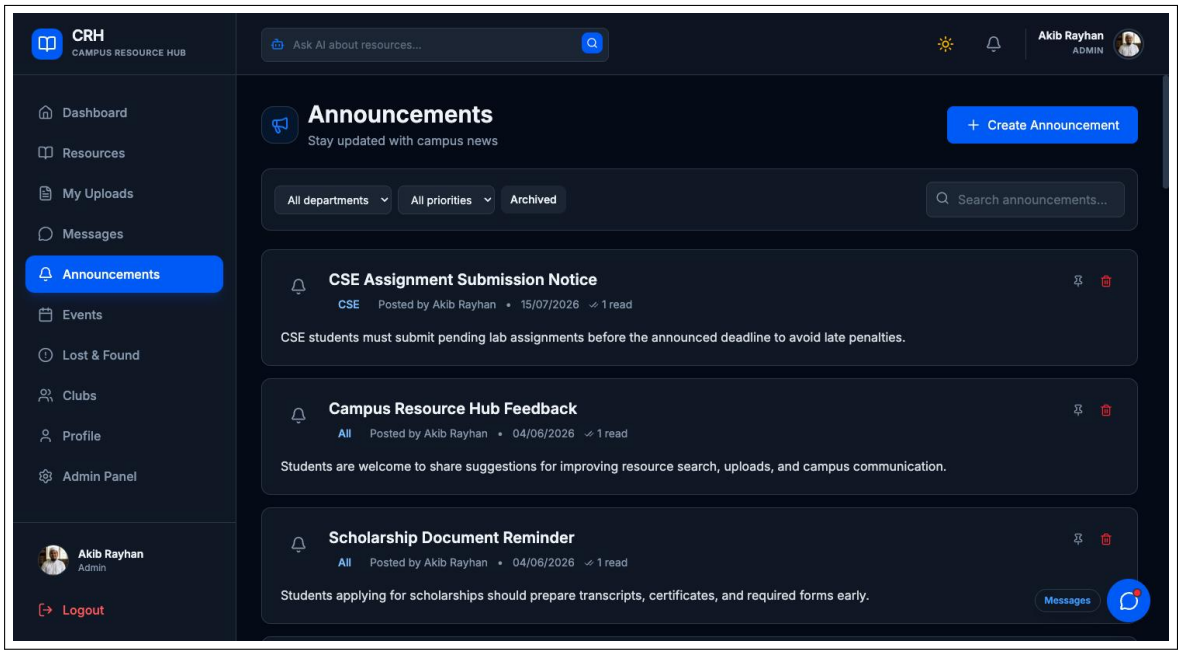

Figure 4.8: Announcement feed with priority badges

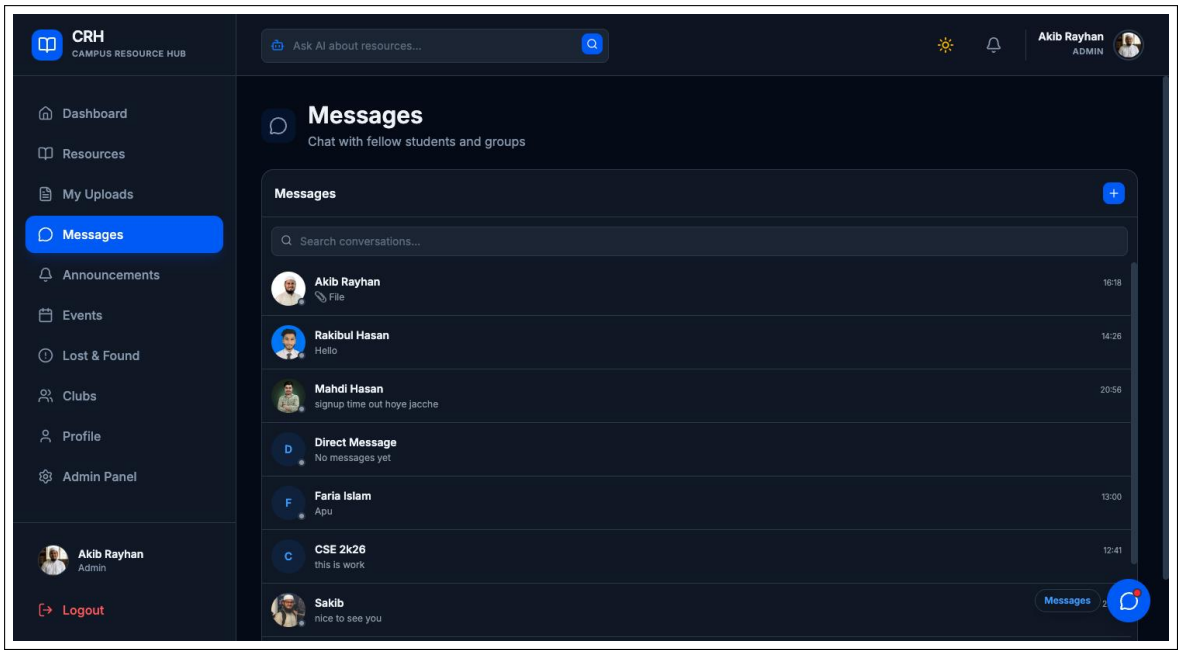

Figure 4.9: Real-time direct and group messaging with seen receipts

4.8 Notifications

Every significant action — new message, approved or rejected upload, event change, claim decision, announcement — creates a notification document and pushes it live over Socket.io. The bell menu supports mark-as-read, mark-all, and clear.

4.9 Semantic Search and AI Assistant

When content is published, its title and body are embedded with a multilingual MiniLM sentence-transformer running inside the Node process (Transformers.js); the 384-dimensional unit vector is stored in the `Embedding` collection, one shared collection for all five content types. A content hash prevents redundant re-embedding when text has not changed. Queries embed the user's question and run Atlas `$vectorSearch`; results are re-checked against each source collection's visibility rules, so search can never leak unpublished content. The AI assistant wraps this pipeline and streams its answer token-by-token over server-sent events. The assistant is reachable from the "Ask AI about resources" bar at the top of every page (visible in Figure 4.1).

For resource files, extracted text and safe image descriptions are split into overlapping ordered chunks and embedded separately in the `ResourceChunk` collection. Retrieval first applies the current user's resource-visibility filter, then ranks matching passages and metadata together. The assistant can therefore answer questions about what a PDF contains, summarize a DOCX file, or describe an indexed image while citing numbered, clickable source cards. Exact-title and lexical-passage matches outrank weaker legacy semantic matches.

4.10 Admin Panel

The panel shows platform statistics (users, resources, pending approvals), a user-management table (block/unblock, delete, role change — role change restricted to admins), and approval queues for every content type including AI-flagged items.

4.11 Deployment

The frontend is deployed on Vercel (global CDN, automatic builds). The backend runs on Render with environment-based configuration; `dns.setDefaultResultOrder("ipv4first")` works around the host's missing IPv6 egress route. MongoDB Atlas hosts the database and the vector search index; Cloudinary stores all files. The whole stack currently runs on free tiers.

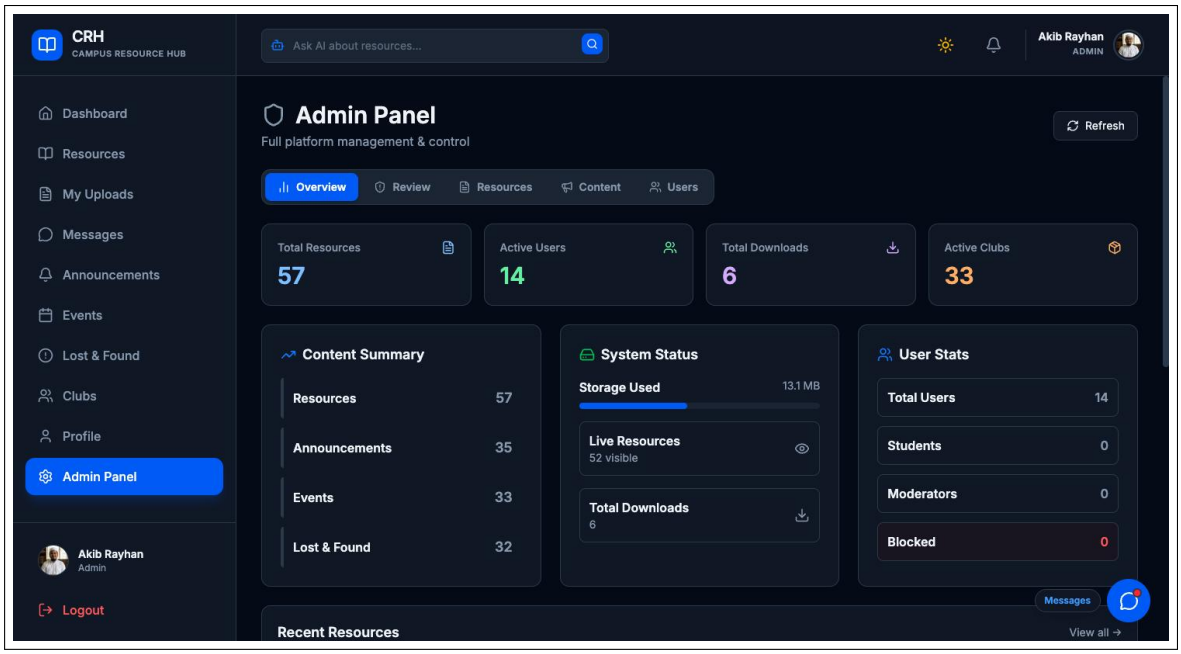

Figure 4.10: Admin panel: statistics and approval queues

Chapter 5: Result & Analysis

5.1 Achieved Features

All objectives set in Chapter 1 were implemented and verified manually across the three roles:

Table 5.1: Feature completion summary

Feature	Status / observed behaviour
OTP signup & JWT login	Accounts created only after correct OTP; expired OTPs auto-deleted by TTL index.
Resource sharing	Upload, preview, download, rating and view counting work for all five file types.
AI moderation	Harmful, partially inspected, or provider-unavailable uploads are quarantined; clean fully inspected files auto-publish.
Approval workflow	Uniform across resources, events, announcements and lost-and-found; rejection reasons delivered as notifications.
Events	Capacity limit enforced at registration time; RSVP status editable; calendar view correct.
Clubs	Open clubs join instantly; request clubs require officer decision; roles enforced.
Lost & found	Contact hidden until claim approval; item lifecycle open → claimed → resolved.
Chat	Real-time delivery with delivered/seen receipts; group administration functions.
Semantic search	Meaning-based matches returned with sources; AI assistant streams answers.
Admin panel	Statistics, user management and all approval queues operational.

5.2 Security Analysis

The two-layer moderation pipeline proved effective in testing: AI-flagged uploads never became publicly visible, while clean and fully inspected uploads were auto-published. Partial extraction or provider failure sent content to human review instead of failing open. Rate limiting blocked request floods with HTTP 429 responses. JWT tampering attempts were rejected by signature verification, and role-guarded endpoints returned HTTP 403 for insufficient roles. Lost-and-found contact details were confirmed to be absent from API responses

until a claim was approved.

5.3 Bug Report and Resolution Log

The Git history of both repositories was reviewed to distinguish implemented fixes from planned work. Table 5.2 records the principal defects that affected safety, search correctness, real-time connectivity, or usability. Commit hashes provide traceable implementation evidence; merged pull requests represent the authoritative state on the `main` branches.

Table 5.2: Resolved defect log derived from Git history and regression tests

ID	Observed defect and impact	Root cause and implemented resolution	Evidence / status
BR-01	Graphic, adult, or otherwise unsafe files could be approved when a vision provider failed or only part of a document was extracted. This was the highest-severity publication risk.	The earlier path treated incomplete inspection too optimistically. Moderation now fails safe: flagged, partial, and provider-unavailable results remain unpublished for human review. Deterministic text screening and the Groq–Hugging Face–OpenAI provider order were also added.	`df5fa96`; four moderation-policy regression tests pass. Resolved.
BR-02	The AI assistant could find a resource by title but could not reliably explain what was inside its PDF or Office document.	Only short metadata and a limited excerpt were searchable. The first repair stored file excerpts (`d356801`); the completed repair adds ordered overlapping `ResourceChunk` passages, embeddings, safe image descriptions, and an existing-resource backfill.	`d356801`, `0e707b3`; 54 approved resources indexed with zero extraction failures. Resolved.

ID	Observed defect and impact	Root cause and implemented resolution	Evidence / status
BR-03	Content questions could rank weakly related legacy semantic matches above an exact title or file-passage match, producing noisy sources.	Metadata and passage relevance had comparable scores. The final hybrid rank gives exact titles and lexical file passages stronger weight, loads ordered passages for summary intent, and preserves inline source numbering.	`0e707b3`; real PDF-summary and deadlock-query checks place the relevant Operating Systems resources first. Resolved.
BR-04	A semantic/vector result risked bypassing the visibility rules used by ordinary keyword queries.	Vector candidates were not sufficient authorization evidence. Every candidate is now joined back through the same approved/owner/role filter before its passages are exposed.	`c755663`, `0e707b3`; streaming test returned zero unauthorized sources. Resolved.
BR-05	Real-time chat connections were unreliable after frontend deployment on Vercel.	The Socket.io client transport did not match the deployed network path. The client now prefers explicit WebSocket transport with deployment-compatible connection settings.	`42ef620`; fix merged to frontend `main`. Resolved.
BR-06	Expanding the header AI input to multiple lines changed the header height and obscured adjacent controls; an interim revert also removed desired behavior.	The textarea participated in normal header layout. It now grows in an absolutely positioned layer, retains Enter/Shift+Enter behavior, and shows readable Markdown plus numbered source cards.	`757141d`, `f4259f7`, `16edb5f`, `641b1bc`; production build passes. Resolved.

5.3.1 Current Verification Snapshot

After the final fixes, the backend unit suite completed with six of six tests passing. End-to-end API checks covered a PDF summary, a topic question across resources, an indexed-image question, and the server-sent-event completion path. The authorization regression test returned eight visible sources and zero unauthorized sources. The frontend production bundle compiled successfully. Browser automation was not part of this snapshot because the dedicated local browser runner was unavailable; this is a tooling limitation, not evidence of a product failure.

5.3.2 Open Operational Constraint

Vision indexing depends on third-party quotas. During the verification run, graphic-image moderation succeeded, but Groq reached a temporary rate limit and the Hugging Face and OpenAI accounts reported exhausted credits. The application handles this condition safely: it does not auto-publish unverified content, and the configured fallback order is retained for the next available provider. This constraint should be monitored separately from functional defects in the application.

5.4 Performance and Capacity Analysis

The deployment currently runs entirely on free tiers, which defines its capacity envelope (Table 5.3).

Table 5.3: Capacity analysis of the current free-tier deployment

Component	Limit	Practical effect
Render web service (free)	512 MB RAM, 0.1 vCPU	Main bottleneck; AI scan + embedding jobs are CPU-heavy
MongoDB Atlas M0	~100 ops/s, 500 connections	Caps simultaneous active users at ~100
Socket.io connections	Few KB per idle socket	Hundreds of idle users are unproblematic
Cloudinary free tier	Monthly bandwidth credits	Long-term quota, not a concurrency limit

In practice the platform comfortably serves **50–100 concurrent active users** (department scale) and can hold **200–300 logged-in connections** including idle users. Load beyond that first saturates the 0.1 vCPU during simultaneous uploads (each triggering an AI scan and embedding generation), then the Atlas M0 operation ceiling. An inexpensive upgrade path

— Render Starter ($7/month) plus Atlas M10 — raises the envelope to 500+ active users, sufficient for full-campus rollout.

5.5 Cost Efficiency

The entire system runs at **zero recurring cost** on free tiers, using Vercel (frontend), Render (backend), MongoDB Atlas M0 (database + vector search), Cloudinary (storage), Brevo (300 e-mails/day), and Groq (moderation API). This makes the platform immediately viable for a university without a software budget.

Chapter 6: Future Enhancements

Planned improvements include:

1. **Mobile application** — a React Native client sharing the existing REST/Socket.io backend.
2. **Background job queue** — moving AI scanning and embedding generation off the request path (e.g. BullMQ) to remove the largest CPU spike.
3. **Push notifications** — Web Push / FCM so users are reachable when the site is closed.
4. **Richer analytics** — per-department dashboards, trending resources, and event attendance reports.
5. **Federated login** — university e-mail SSO (OAuth2) alongside OTP signup.
6. **Horizontal scaling** — multiple backend instances with a Socket.io Redis adapter once campus-wide load requires it.
7. **Accessibility and localization** — full Bangla interface and WCAG-compliant UI review.

Chapter 7: Conclusion

This project set out to replace the fragmented, unmoderated channels through which campus information currently flows with a single, secure, intelligent platform. The delivered system — Campus Resource Hub — implements verified onboarding, a categorized and rated study-resource repository, capacity-aware events, club management, a privacy-preserving lost-and-found process, scheduled announcements, real-time messaging, and a semantic AI assistant, all protected by a uniform two-layer moderation pipeline and role-based access control. The implementation demonstrates that modern techniques usually associated with large commercial platforms — AI content moderation, vector-based semantic search, streamed LLM assistance — can be integrated into a student project deployed entirely on free cloud tiers. Capacity analysis confirms the current deployment already serves a department-sized community, with a clear and inexpensive path to campus scale. Beyond its immediate utility, the project provided deep practical experience in full-stack architecture, real-time systems, applied AI services, and cloud deployment.

Bibliography

[1] Meta Platforms, "React – The library for web and native user interfaces," `https://react.dev`, accessed 2026.

[2] OpenJS Foundation, "Express – Node.js web application framework," `https://expressjs.com`, accessed 2026.

[3] MongoDB Inc., "MongoDB Atlas Vector Search documentation," `https://www.mongodb.com/docs/atlas/atlas-vector-search/`, accessed 2026.

[4] Socket.io, "Socket.io documentation," `https://socket.io/docs/`, accessed 2026.

[5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of EMNLP-IJCNLP*, 2019.

[6] W. Wang et al., "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[7] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015.

[8] Cloudinary Ltd., "Cloudinary developer documentation," `https://cloudinary.com/documentation`, accessed 2026.

[9] Moodle HQ, "Moodle – Open-source learning platform," `https://moodle.org`, accessed 2026.